

7. 容器技术

冯奕
2026



南京大學
NANJING UNIVERSITY

目录

- 容器基本概念
- Docker
 - 镜像
 - 容器和镜像的关系
- 容器编排
 - 动机、定义
 - 工作流程
 - 实现
 - 优点
- Kubernetes
- 移动云容器服务案例



容器是什么？

- 基于操作系统内核的轻量级虚拟化技术
- 将应用及其相关运行时环境打包成一个可移植、可部署的单元（镜像）
- 在不同的环境中以独立且可重复的方式运行



与虚拟机的区别

- 虚拟机技术：抽象并虚拟化计算机物理资源
 - 模拟计算机的整个物理基础设施（如磁盘、驱动器和服务器设备）
 - 操作系统级别的资源隔离
- 容器技术：更轻量级的虚拟化技术。
 - 模拟操作系统，而非硬件
 - 进程级别的资源隔离



核心优势

- 轻量
 - 只包含了应用与必要的依赖
 - 迁移、部署方便
 - 快速启动：无需经历开机流程
 - 运行所需硬件资源远少于虚拟机
- 硬件兼容性高
 - 不依赖 CPU 的硬件虚拟化支持
 - 借助操作系统内核机制就可以实现必要的隔离



与虚拟机的区别

特性	虚拟机	容器
虚拟化层级	硬件级虚拟化 (Hypervisor)	操作系统级虚拟化 (共享内核)
隔离性	强隔离 (独立操作系统)	进程级隔离 (依赖Linux内核机制)
启动速度	分钟级 (需启动完整OS)	秒级 (直接调用主机内核)
资源占用	高 (每个VM需独立OS和驱动)	低 (共享内核, 无冗余开销)
镜像大小	GB级 (包含完整OS)	MB级 (仅应用和依赖)



容器为什么要考虑隔离？

- 容器隔离是为了安全考虑
- 隔离指：
 - 限制容器使用资源的量，如GPU、内存等，防止不当竞争
 - 避免容器对操作系统的修改影响到其他容器，例如对主机进程、网络、用户等信息的修改
 - 避免并发导致的不一致性，例如保证在容器中的文件系统读写不影响到其他容器
- 不安全：容器内的恶意软件从容器中逃离，攻击主机或其他容器



如何实现容器隔离？

- 资源隔离（1）
 - 利用操作系统内核
 - 1. 例如linux内核的cgroup特性：允许创建不同的进程控制组，并限制、记录和隔离进程组使用的物理资源，包括CPU、内存、I/O等
 - 四大核心组件：任务、控制组、层级、子系统
 - 通过使用控制组，容器可以保证在资源有限的情况下，应用程序能够稳定运行



如何实现容器隔离？

- 资源隔离（2）
 - 利用操作系统内核
 - 2. 例如linux内核的Namespace特性：不同的namespace中的程序可以享有一份独立的系统资源
 - PID、User、Mount（文件挂载点）、IPC（进程通信）、UTS（主机名和域名）、Network（网络）
 - 通过使用namespace，容器可以隔离出一个独立的运行环境，使得容器内的进程看不到或不能访问宿主机或其他容器的资源



Cgroups vs. Namespace

特性	Namespace	cgroups
核心目标	隔离资源视图（让进程“看不到”彼此）	限制资源用量（控制进程“用多少”）
实现机制	修改进程对系统资源的可见性	设置资源配额（CPU/内存/磁盘等）
典型应用	隔离进程、网络、文件系统	限制 CPU、内存使用量，分配优先级
功能侧重点	环境隔离（安全性、独立性）	资源管控（公平性、稳定性）



如何实现容器隔离？

- 内核隔离
 - Windows容器——Hyper-V隔离模式
 - 它可以提供硬件级别的隔离，让每个容器都有自己的内核。
 - 这种隔离模式可以提供更高的安全性，并且可以提供更好的兼容性



与普通容器（进程级）的区别

特性	Hyper-V隔离模式	进程隔离模式
隔离级别	硬件级（虚拟机级）	进程级（共享宿主机内核）
安全性	高（独立内核，防止逃逸攻击）	较低（依赖宿主机内核安全性）
启动速度	秒级（比VM快，比进程隔离慢）	毫秒级（极快）
资源占用	较高（需分配虚拟硬件资源）	极低（共享宿主机资源）
内核兼容性	支持不同版本内核	必须与宿主机内核版本一致



目录

- 容器基本概念
- Docker
 - 镜像
 - 容器和镜像的关系
- 容器编排
 - 动机、定义
 - 工作流程
 - 实现
 - 优点
- Kubernetes
- 移动云容器服务案例



容器引擎Docker

- Docker 最初是 dotCloud 公司（Docker 公司的前身）在法国期间发起的一个公司内部项目，后来以 Apache 2.0 协议开源，代码在Github上进行维护
- Docker的核心组件：
 - Docker Daemon：后台服务，负责管理镜像等。
 - Docker Client：命令行工具，与Daemon交互
 - Registry：镜像仓库
- Docker 的核心技术：
 - 基于 namespace 和 cgroup 实现进程和资源隔离
 - 基于写时复制的镜像技术



一个具体的产品——推动了容器技术发展

- Docker 通过提供一个简单易用的工具，使得开发者可以方便地创建和管理容器。
- Docker 的出现推动了容器技术的发展，基于它引入了容器编排工具，如 Docker Swarm、Kubernetes、Mesos、Rancher 等用于管理大规模的容器
- Docker提供容器运行的环境，容器运行的的是一个镜像



什么是镜像？——容器的载体

- 镜像(Image) 是一个只读的容器模板，它包含了运行应用程序所需的所有文件系统、库、
作是容器的静态快照或模板。

```
# 1. 使用官方Nginx镜像（alpine版本，体积最小）
```

```
FROM nginx:alpine
```

```
# 2. 删除默认的Nginx欢迎页面（可选）
```

```
RUN rm -rf /usr/share/nginx/html/*
```

- 在Docker中，镜像是通过 Dockerfile 构建而成的。

```
# 3. 将本地网页文件复制到容器中  
COPY ./website /usr/share/nginx/html
```

```
# 4. 暴露80端口（Nginx默认端口）
```

```
EXPOSE 80
```

- Dockerfile 中描述了构建镜像的所有步骤。



Dockerfile构建 vs. 已有容器构建

维度	从已有容器构建 (docker commit)	从Dockerfile构建
操作方式	手动启动容器 → 进入容器修改 → 提交容器为新镜像	编写Dockerfile → 通过docker build自动执行构建步骤
依赖文件	不需要Dockerfile, 依赖容器运行状态	必须编写Dockerfile文件
构建命令	docker commit <容器ID> <新镜像名>	docker build -t <镜像名> .
构建过程	基于容器的实时状态生成镜像 (黑盒操作)	按Dockerfile指令逐层执行 (透明化操作)



Dockerfile构建 vs. 已有容器构建

特性	从已有容器构建	从Dockerfile构建
可重复性	✗ 依赖手动操作，易导致镜像不一致	✓ 声明式步骤，确保每次构建结果一致
透明性	✗ 修改过程无记录，无法追溯具体变更	✓ 构建步骤完全可见，可通过docker history逐层检查
镜像体积	✗ 可能包含临时文件、未清理的缓存，导致体积臃肿	✓ 支持多阶段构建和缓存清理，优化体积
维护性	✗ 难以维护，修改需重新手动操作	✓ 修改Dockerfile即可更新镜像，易于版本控制和协作
安全性	✗ 可能包含敏感操作（如密码硬编码），难以排查	✓ 可通过分层检查漏洞，支持非特权用户运行



什么是镜像？——容器的载体

- 镜像的特点

- 不可修改性：镜像是只读的，一旦创建，就无法更改。
- 分层存储：镜像是通过多个文件系统层（Layer）组成的，不同的镜像可以共享相同的层从而节省存储开销。
- 版本控制：镜像可以通过标签（Tag）进行版本控制，可以基于不同的标签部署和管理应用程序的不同版本。



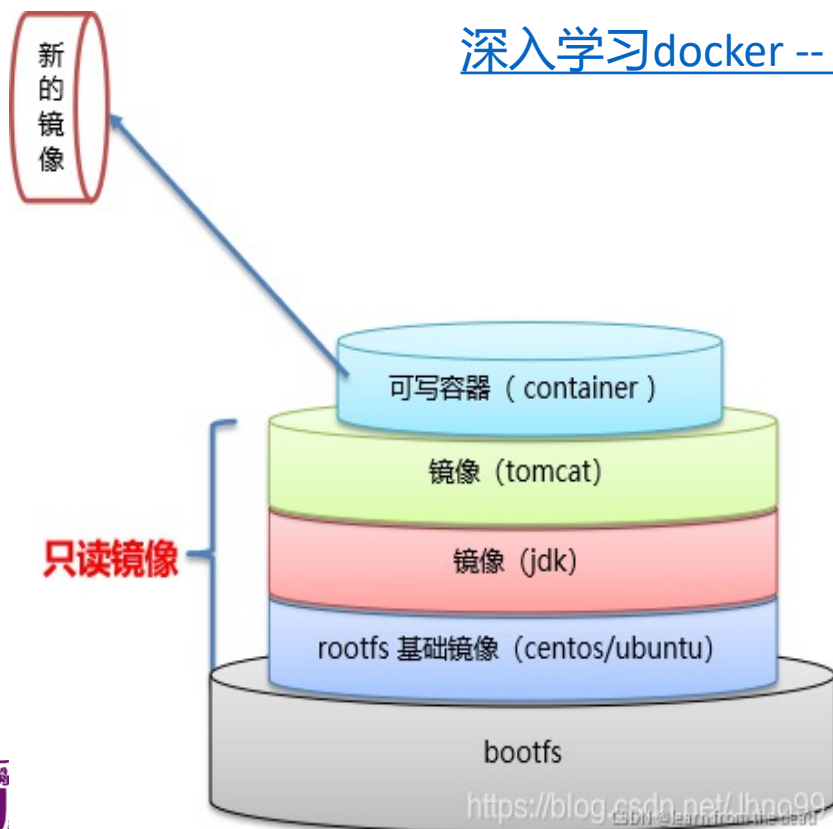
镜像的构建

- base镜像: build from scratch
 - 镜像与宿主机的操作系统是共用一个内核和bootfs
 - base镜像不基于其他镜像, 而是直接基于宿主机的内核和bootfs构建。
 - 因此镜像和宿主机共享内核, 因此容器是一种操作系统级的虚拟化
- 扩展镜像:
 - 基于一个已有的镜像 (FROM <image>)
 - 逐步构建出一个新的镜像, 每一步 (Step) 都会形成一个镜像层 (Layer)
 - 构建镜像的步骤可以通过Dockerfile描述



镜像的具体实现

- 镜像分层的机制：写时复制（Copy On Write） + UnionFS
- 镜像的构建：从 bootfs 开始

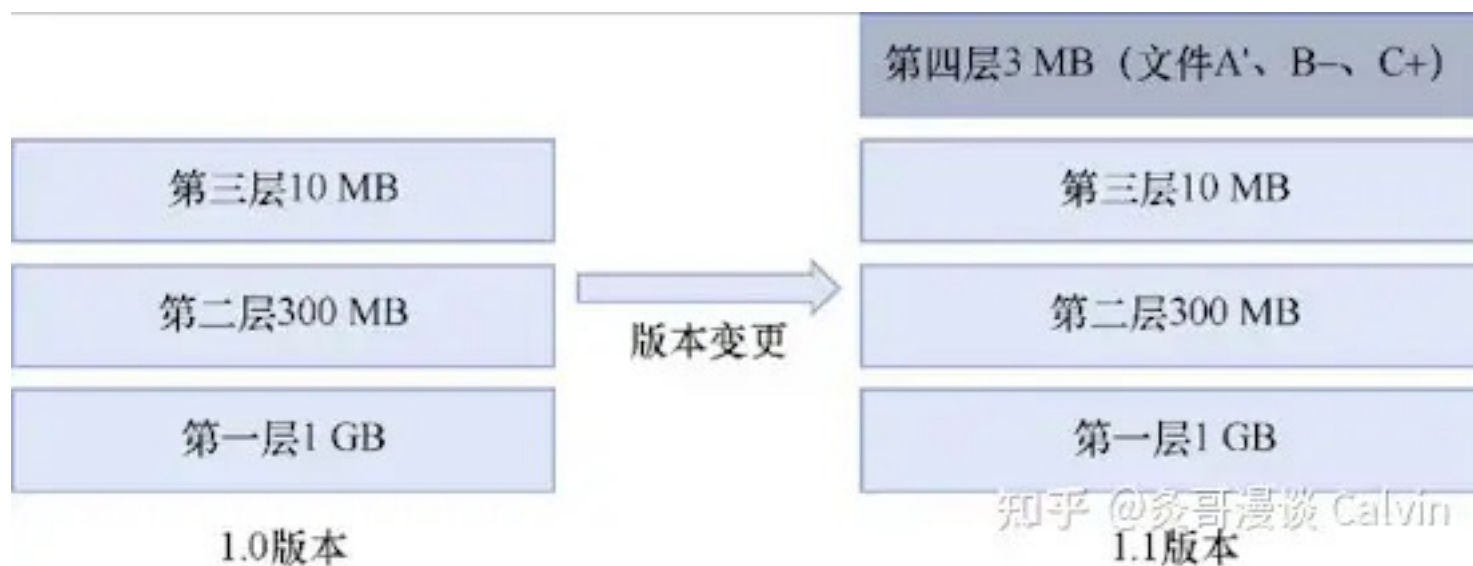


[深入学习docker -- 联合文件系统 OverlayFS - 知乎 \(zhihu.com\)](#)

通过将镜像进行分层，尽可能复用层，减少镜像存储空间，提高镜像拉取速度。

镜像的具体实现

• 镜像的变更



构建镜像加速方式

- 缓存构建
 - Docker 利用分层存储的特性，在构建镜像时复用未变化的层，跳过重复操作以加速构建流程。
- 多阶段构建
 - 在单个 Dockerfile 中定义多个构建阶段（使用多个 FROM 指令），每个阶段独立运行，最终仅保留必要文件，显著减小镜像体积。
 - 阶段1：使用完整工具链编译代码。
 - 阶段2：仅复制编译产物到轻量级运行环



容器和镜像的关系

- 每个容器都是基于镜像来运行的，它们共享相同的镜像文件系统，但具有独立的写入层，即容器层（Container Layer）。这使得多个容器可以在相同的基础镜像上运行，同时保持各自的独立性。
- 容器层依然保有写时复制的特性，由于容器层记录的信息和镜像的每一层记录的信息是相同的，因此可以直接将容器层+镜像提交为一个新的镜像。
- 容器与镜像分别刻画了同一个对象的动态与静态；镜像可以启动容器，运行中的容器也可以导出为镜像。二者的底层存储技术都是 UnionFS 技术



目录

- 容器基本概念
- Docker
 - 镜像
 - 容器和镜像的关系
- 容器编排
 - 动机、定义
 - 工作流程
 - 实现
 - 优点
- Kubernetes
- 移动云容器服务案例



什么是编排、为什么要编排？（1）

- 生产中，需要管理运行应用容器，以确保服务不会下线
 - 例如，当一个容器发生故障时，你可能选择重新启动该容器；当业务量发生变化时，可能需要对应用的容器数量进行动态扩容，也就是增加或减少启动的容器数量。
- 当数量较少时，容器可以轻松手动部署和管理。但在大多数企业，容器化应用程序的数量以及业务量正在快速增长，靠手动难以管理

这就是容器编排



- 如果这些行为交由给系统做自动化处理处理，显然会更容易



什么是编排、为什么要编排？ (2)

- 容器编排是对大量容器进行自动化集中管理和调度的工具和技术
- 如果没有自动化编排框架，是非常难对大规模容器化应用程序进行管理的，尤其是作为持续集成/持续部署 (CI/CD) 或 DevOps 管道的一部分的时候。
- 确保资源的高效利用，提高应用的可用性和性能



容器编排-工作流程

- 虽然不同工具的方法和功能存在差异，但容器编排本质上都具有如下三步过程
 - 开发人员编写配置文件，定义应用的目标状态
 - 编排工具选择最优主机，并部署容器
 - 编排工具管理容器生命周期，监控容器运行状态
- 编排前、编排后还需要完成的工作
 - 编排前——设计容器化应用程序的架构
 - 编排后——评估并优化编排策略



容器编排-配置

- 配置文件：JSON或者YAML（Yet Another Markup Language）
 - 定义应用程序的镜像，以及镜像所在的仓库地址
 - 定义容器副本数
 - 定义部署容器的节点选择策略
 - **亲和性**——和相关的容器或节点一起运行
 - **反亲和性**——与相关的容器或节点不在一起运行
 - 定义容器的 CPU、内存等资源限制
 - 定义容器失效时的处理策略
 - 定义容器的网络连接
 - 指定版本控制（用于分阶段或金丝雀发布）

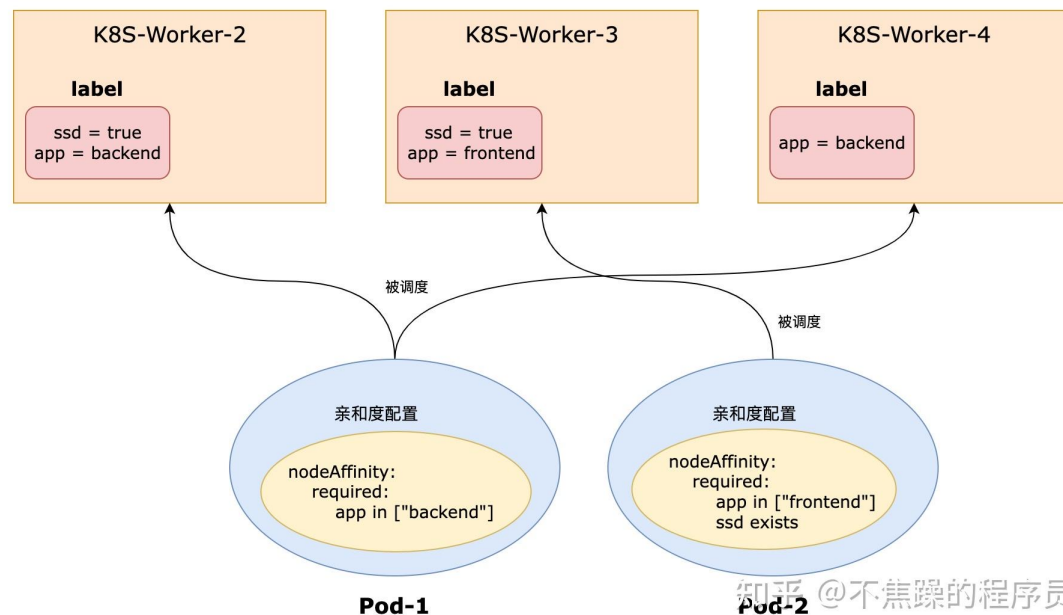


容器编排-亲和性

- Node（运行容器的节点）亲和性、Pod（容器组）亲和性；

被调度的都是Pod

- Node亲和性调度是指Pod和Node的亲密关系。
 - Pod亲和性调度是指Pod和Pod的亲密关系。
 - 硬亲和：亲和规则只有一种，必须符合该规则。
 - 软亲和：规则有多种，每个权重不同，根据权重优先级去选择一个规则。
- Node亲和性调度：



Dockerfile构建 vs. 已有容器构建

特性	节点亲和性 (Node Affinity)	Pod 亲和性 (Pod Affinity)
目标对象	节点 (Node)	其他 Pod
核心场景	硬件/位置依赖、资源类型选择	服务协同、高可用性、资源隔离
硬性规则示例	必须部署到 GPU 节点	必须与缓存服务部署在同一节点
软性规则示例	优先使用 SSD 节点	尽量不与监控服务部署在同一节点



容器编排-调度

- 开发人员完成配置文件的编写后，编排工具应用该配置文件来调度并部署配置的容器。
- 通常需要根据主机上剩余可用的 CPU 容量、内存，以及配置文件中的资源申请或其他约束，来选择部署容器的最佳主机。
- 选择完成后，在节点上自动拉取镜像并启动容器



容器编排-管理与监控

- 活动包括：
 - 容器失效时，重新启动容器
 - 节点失效时，重新调度容器
 - 滚动更新：有新版本的镜像时，启动新的容器并成功运行后，再中断旧的容器
 - 管理容器的可扩展性（向上和向下）【动态扩缩】、负载均衡和资源分配
 - 收集和存储用于应用程序运行状况、日志等数据



容器编排-优势-再一次

- 自动化：
 - 减少重复性任务，消除安装、管理和维护容器的负担
 - 降低了管理大型容器化应用程序的工作量和复杂性
 - 支持敏捷开发或 DevOps 方法，实现快速迭代
- 高资源利用率：根据配置选择最优主机，提高计算资源利用率
- 高可扩展性：方便扩大或缩小容器部署
- 高可用性：实时监控容器运行状态
 - 容器失效：重启
 - 主机失效：迁移
- 增强的安全性：自动化管理安全策略，减少人为漏洞



目录

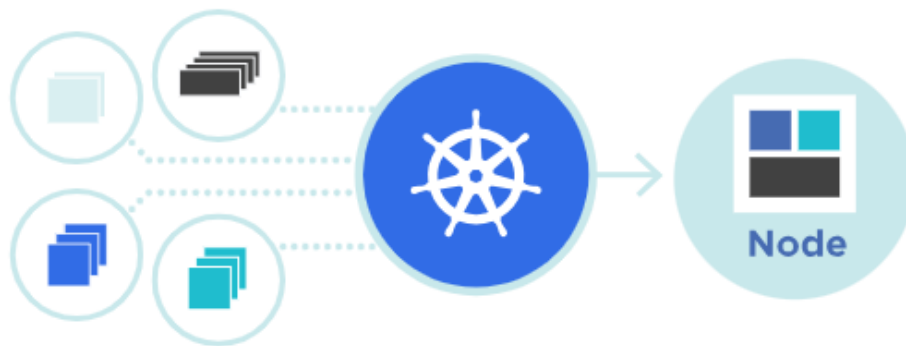
- 容器基本概念
- Docker
 - 镜像
 - 容器和镜像的关系
- 容器编排
 - 动机、定义
 - 工作流程
 - 实现
 - 优点
- Kubernetes
 - 基础
 - 生态+实现
- 移动云容器服务案例



Kubernetes——简介



- 也称K8S, 开源
- 一个可弹性运行的分布式系统的框架
- 用于自动部署、扩缩和管理容器化应用程序; Kubernetes 会满足你应用的扩展要求、故障转移、提供部署模式等
- 源自 Google 15 年生产环境的运维经验, 同时凝聚了社区的最佳创意和实践



Kubernetes——核心思想

- 我们常说 k8s 是使用最广泛的容器编排平台，事实上成为了容器编排的实践标准。k8s 官方认为：Kubernetes 不仅仅是一个编排系统，实际上它消除了编排的需要。
- 编排技术的定义是**执行已定义的工作流程**：首先执行 A，然后执行 B，再执行 C。更熟悉的术语来说，**编排是面向过程的**，用户需要自定义编排过程。
- 而 Kubernetes 不是面向过程，而是包含了一组独立可组合的控制过程，**可以持续地将当前状态驱动到所提供的预期状态**。
- 用户不需要在乎如何从 A 移动到 C，也不需要集中控制，**中间过程对用户透明**。
- 这使得系统**更易于使用且功能更强大、系统更健壮，更为弹性和可扩展**。



Kubernetes——特性

自动化上线和回滚

自动装箱

服务发现与负载均衡

批量执行

自我修复

IPv4/IPv6 双协议栈

存储编排

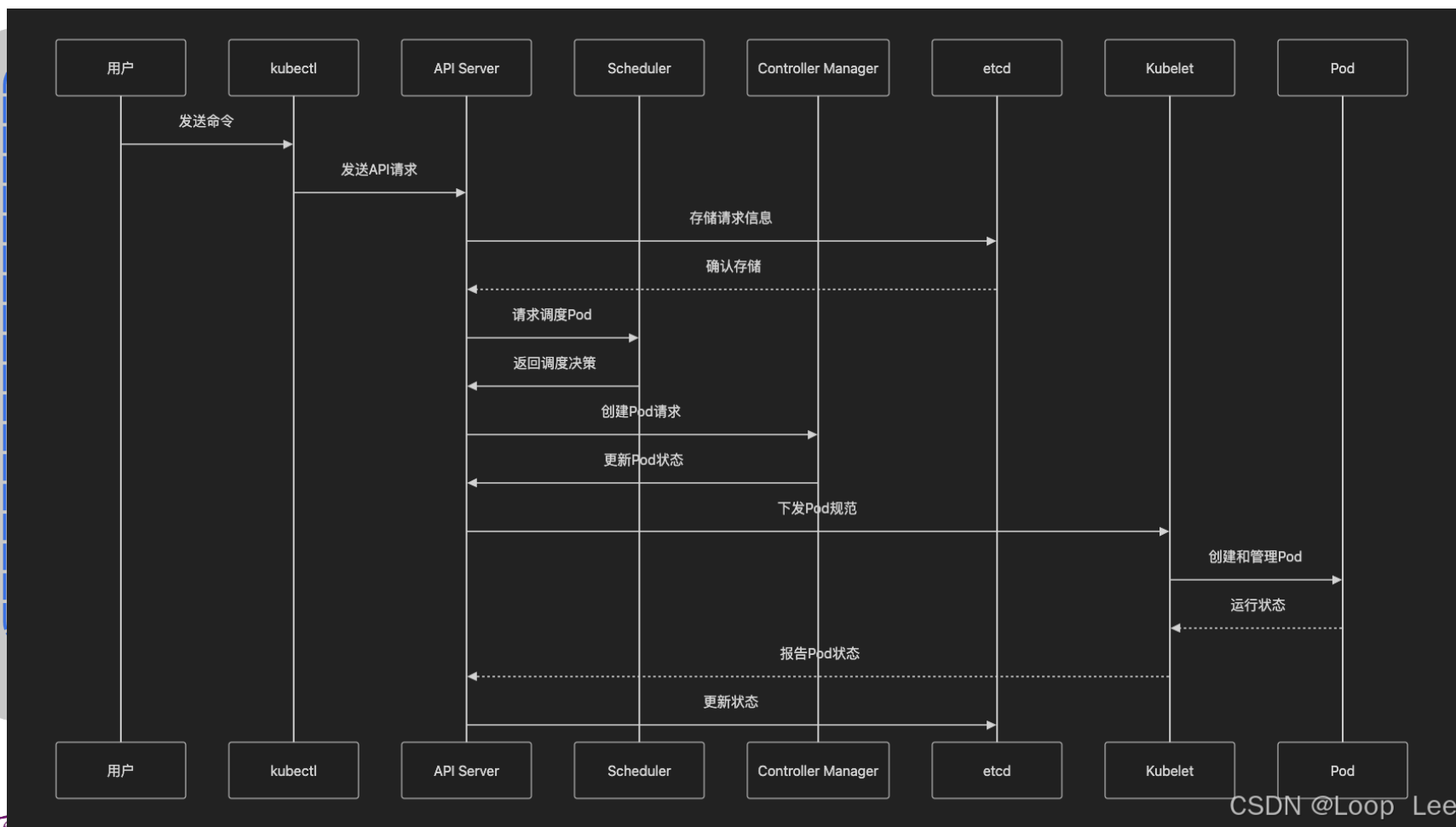
水平扩缩

Secret 和配置管理

为扩展性设计



Kubernetes——组件



CSDN @Loop Lee



Kubernetes——Pod

- Pod 对特定于应用程序的“逻辑主机”进行建模：它包含一个或多个相对紧密耦合的应用程序容器。
- Pod 是一组（一个或多个）容器，始终被共同调度以位于同一位置（节点/主机）上，并在共享上下文中运行
 - 共享 Linux Namespace、cgroup
 - 共享文件系统卷
 - 共享网络
- Pod 是 k8s 中创建和管理的最小的可部署计算单元



Kubernetes——Pod声明示例

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx
spec:
  containers:
  - name: nginx
    image: nginx:1.14.2
    ports:
    - containerPort: 80
```

一个最基础的 Pod 配置文件，该文件声明了一个名为 nginx 的 Pod，该 Pod 包含一个容器，容器名为 nginx，使用镜像为 nginx:1.14.2，容器暴露的端口为 80



Kubernetes——命令行 (1)

- `kubectl [command] [TYPE] [NAME] [flags]`
- Command:
 - create: 创建资源
 - get: 查看资源
 - describe: 查看资源在 k8s 中的详细配置
 - delete: 删除资源
- TYPE:
 - pod
 - 工作负载: deployment/deploy, daemonset/ds, statefulset/sts,
 - services/svc, serviceaccounts/sa
 - pv, pvc
 - ConfigMap, Secret



Kubernetes——命令行 (2)

- `kubectl [command] [TYPE] [NAME] [flags]`
- NAME: 某个具体资源的名称
- flags:
 - -n: 指定命名空间
 - -A: 所有命名空间
 - --sort-by: 指定排序方法
 - -o JSON | YAML: 指定输出格式
 - -o JSONPATH: 指定 JSON 输出时, 输出哪些字段



Kubernetes——命令行（3）——示例

- 应用 YAML 配置文件： `kubectl apply -f <配置文件>.yaml`
 - 通常一个应用包括很多种资源，对应很多个 YAML 文件。
 - 如果需要运维人员手动执行 `kubectl apply` 命令，十分繁琐。
可以使用 Helm 来部署应用。
- 创建命名空间： `kubectl create ns <命名空间>`
- 查询某个命名空间下所有 Pods： `kubectl get pods -n <命名空间>`
- 查看某个 Pod 详细信息（调度状态、资源申请状况）：
`kubectl describe po -n <命名空间> <Pod 名>`



Kubernetes——工作负载

- 生产中，通常不会直接创建 Pod。因为虽然 Pod 声明配置中有重启策略，但是无法实现动态扩缩容、滚动更新等编排需求。
- 为了实现上述目标，应该配置 workload 工作负载。在工作负载中可以声明 Pod 的目标状态
 - Deployment 针对无状态应用的工作负载，无状态意味着所有 Pod 之间没有区别，每一个 Pod 都提供相同的服务，可以相互替换
 - DaemonSet 针对那些需要运行在每个节点上的后台应用。当集群中新增节点时，会自动在新节点上创建 DaemonSet 中的 Pod
 - StatefulSet 针对有状态应用的，因为 Pod 是不可互换的，通常要求稳定的、唯一的网络标识符，以及稳定的、持久的存储。



Kubernetes——负载示例—Deployment

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.14.2
          ports:
            - containerPort: 80
```

- apiVersion:** 这是创建 Deployment 所需的 Kubernetes API 版本。
- kind:** 这表示你要创建的资源类型，这里是 Deployment。
- metadata:** 这部分包含了 Deployment 的元数据，包括名称。
- spec:** 这部分包含了 Deployment 的规格，包括副本数量、选择器、模板等。

这个配置文件告诉 Kubernetes 希望在集群中运行 3 个 nginx Pod，并且当这些 Pod 失败时，Kubernetes 会自动创建新的 Pod 来替换它们。



Kubernetes——Service

- k8s 中的 Service API 资源允许用户在不修改现有应用程序的前提下，就能利用 k8s 的服务发现机制，并对集群内部/外部提供服务。
- Service 一共有四种类型（type）：
 - 1. ClusterIP：（k8s 集群通常使用网络插件为 Pod 分配集群内部的 IP 地址），Service 通过集群的内部 IP 并只能在集群内部访问。没有服务显式指定 type 时的默认值。
 - 2. NodePort：通过每个节点上的 IP 和静态端口（端口号从30000开始）公开 Service，类似于 Docker 的端口映射允许将容器端口映射到宿主机端口。集群外部可以通过节点IP+端口访问服务
 - 3. LoadBalancer：使用云平台的负载均衡器向外部公开 Service。
Kubernetes 不直接提供负载均衡组件；你必须提供一个，或者将你的 Kubernetes 集群与某个云平台集成。
 - 4. ExternalName：负责将外部的服务在集群内部进行暴露



Kubernetes——Service——示例

```
apiVersion: v1
kind: Service
metadata:
  name: nginx
  labels:
    app: nginx
spec:
  type: NodePort
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30001
  selector:
    app: nginx
```

本例中创建了一个名为 `nginx` 的 `Service` 资源，集群内的容器可以通过 `url:nginx.<namespace>` 来访问该服务。外部通过 `<ip, port>` 访问。

`Service` 类型被设置为 `NodePort`，因此可以将 `Deployment` 所创建的 `Pod` 的 `80` 端口映射到集群每个节点的 `30001` 端口，集群外部可以通过节点 `IP+端口号` 访问到 `nginx` 的服务。



Kubernetes——持久化存储卷

- 由于容器的隔离性，容器内部的数据会保存在容器的可写层中，不会影响到宿主机，而容器删除后，这些内容也一并被删除了。
- Docker 提供存储卷机制，允许用户将宿主机上的某个目录作为存储卷挂载到容器的文件系统上，这样容器在该存储卷上保存的数据相当于直接写到了宿主机上。
- 存储卷与容器之间是松耦合的：
 - 容器删除后，存储卷不会消失，因为本身只是宿主机上某个目录而已
 - 方便容器之间共享存储卷
 - 方便迁移
 - 存储卷可以放在远程主机、云服务提供商
- 在 Kubernetes 中，用户可以使用 Persistent Volume (PV) 和 Persistent Volume Claim (PVC) 来声明和管理存储卷。PV 是集群中的一块存储，它是一个资源对象，而 PVC 是对 PV 的请求，它是一个请求存储资源的方式。
- PVC 通过在软件设计模式的**命令模式**，将对 PV 的请求封装为 k8s 中的一个资源对象。



Kubernetes——PV和PVC声明配置 (1)

•场景:

- 管理员创建 PV
- 用户创建 PVC, k8s 会自动选择合适的 PV 绑定到 PVC 上
- 用户创建 Pod, 声明使用 PVC

集群管理员声明了一个名为 my-pv 的 PV。存储容纳量为 1GiB, 访问模式为 ReadWriteOnce, 即只能挂载到一个 Pod 上。

如果声明为 ReadWriteMany, 则可以在多个 Pod 中进行共享。

hostPath 指定了存储的实际位置在宿主机的 /var/my-pv 目录下。类似于 Docker 的 Volume, 必须指定宿主机的某个目录。

定义了 StorageClass 的名称为 manual。它将用于将 PersistentVolumeClaim 的请求绑定到此 PersistentVolume。

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: my-pv
spec:
  storageClassName: manual
capacity:
  storage: 1Gi
accessModes:
  - ReadWriteOnce
hostPath:
  path: "/var/my-pv"
```



Kubernetes——PV和PVC声明配置 (2)

定义了一个名为 my-pvc 的 Persistent Volume Claim，它请求 1Gi 的存储空间，访问模式为 ReadWriteOnce。

K8s 会寻找相同 storageClassName 以及满足容量需求的 PV 将其绑定到 PVC 上

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: my-pvc
spec:
  storageClassName: manual
  accessModes:
    - ReadWriteOnce
resources:
  requests:
    storage: 1Gi
```



Kubernetes——Pod 声明 PV+PVC

- 区分 3 个绑定关系：PV-PVC, PVC-Pod, PVC/PV-Container

spec:

containers:

- name: my-container

image: nginx

volumeMounts:

- mountPath: "/usr/share/nginx/html"

name: my-pvc

volumes:

- name: my-pvc

persistentVolumeClaim:

claimName: my-pvc

这里省略了 apiVersion 和 metadata 等配置

Pod 使用 PVC 的 name 声明使用哪个 PVC，并在容器配置中声明将 /usr/share/nginx/html 目录作为挂载点，这样，容器可以访问该路径，就等价于访问到存储在 Persistent Volume 上的数据。

容易混淆的地方：Volumes 和 volumeMounts:

Volumes 配置了 Pods 使用的存储卷，使用 PVC 的名称进行声明；volumeMounts 配置了容器挂载存储卷，同时声明需要使用的 PVC 的名称，以及容器内部的挂载点 mountPath



目录

- 容器基本概念
- Docker
 - 镜像
 - 容器和镜像的关系
- 容器编排
 - 动机、定义
 - 工作流程
 - 实现
 - 优点
- Kubernetes
 - 基础
 - 生态+实现
- 移动云容器服务案例



Kubernetes生态——镜像仓库

- Harbor 是由 VMware 开发的一个开源的 Docker 镜像仓库，用于存储和分发 Docker 镜像。提供了一个图形界面，便于管理 Docker 镜像，包括：上传、下载、删除等操作。
- Harbor 还支持角色管理、访问控制、漏洞扫描、镜像签名等高级功能。
- Harbor 和 Docker Hub 的主要区别在于：
 - Harbor 是私有的，只有被授权的用户才能访问和操作镜像，这可以提高安全性。而 Docker Hub 是公共的，任何人都可以访问和操作镜像，这可能会带来安全风险；可能会受到网络延迟、服务中断等因素的影响。
 - Harbor 提供了一些 Docker Hub 没有的高级功能，如角色管理、访问控制、漏洞扫描（现在 Docker 也支持了）、镜像签名等。



Kubernetes生态——应用管理（1）

- 当应用包含大量的 k8s 资源，对应许多声明文件时，K8s 部署应用中会遇到如下问题：
 - 每个声明文件都需要 apply 到集群中
 - 版本回滚、升级时，删除已有资源，重新创建资源
 - 资源之间还有依赖顺序
 - 不同环境下部署应用需要不同的配置，需要修改声明文件中某些值
- Helm：管理 k8s 应用的工具。使用名为 Helm Chart 的压缩包描述应用，包含了一组 Kubernetes 资源定义的文件。



Kubernetes生态——应用管理（2）

- 描述了如何在 k8s 集群中部署一个应用：
 - Chart.yaml 描述了 Helm Chart 包的元数据，如：名称、版本
 - templates/ 目录存放模板文件，用于生成 k8s 资源定义文件
 - values.yaml 模板文件的默认配置
 - resources/ 目录存放资源文件，如：ConfigMap, Secret, PVC 等
 - charts/ 目录包含了 Helm Chart 的依赖
- helm 命令行工具，实现应用的安装、升级、回滚等
- 支持使用 Helm Repository 来分享和获取 Helm Chart



Kubernetes生态——其他

- Prometheus：是一个开源的监控和警报工具包，可以用来收集和存储 Kubernetes 集群和应用的指标。Prometheus 可以与 Grafana 等工具集成，提供丰富的图表和可视化界面。
- Fluentd：一个开源的数据收集器，可以用来收集 Kubernetes 集群和应用的日志。以与 Elasticsearch 等工具集成，提供丰富的日志分析和搜索功能。



Kubernetes实现——GPU资源管理

- Kubernetes 管理和分配集群中 GPU 资源的必要性：AI 应用的容器化趋势
- 需要多组件配合：
 - K8s Scheduler Plugin：提供面向GPU资源的增强调度功能（GPU设备选择、拓扑感知调度等）
 - K8s Device Plugin：向集群暴露节点上的GPU设备资源
 - K8s CRD + Operator：监控节点 GPU 状态，为调度器和故障转移提供数据支持
 - Container Runtime：为容器使用节点上的GPU设备提供底层支持（一般包括设备文件和驱动库的挂载）
 - Virtualization Support：为 GPU 虚拟化提供支持



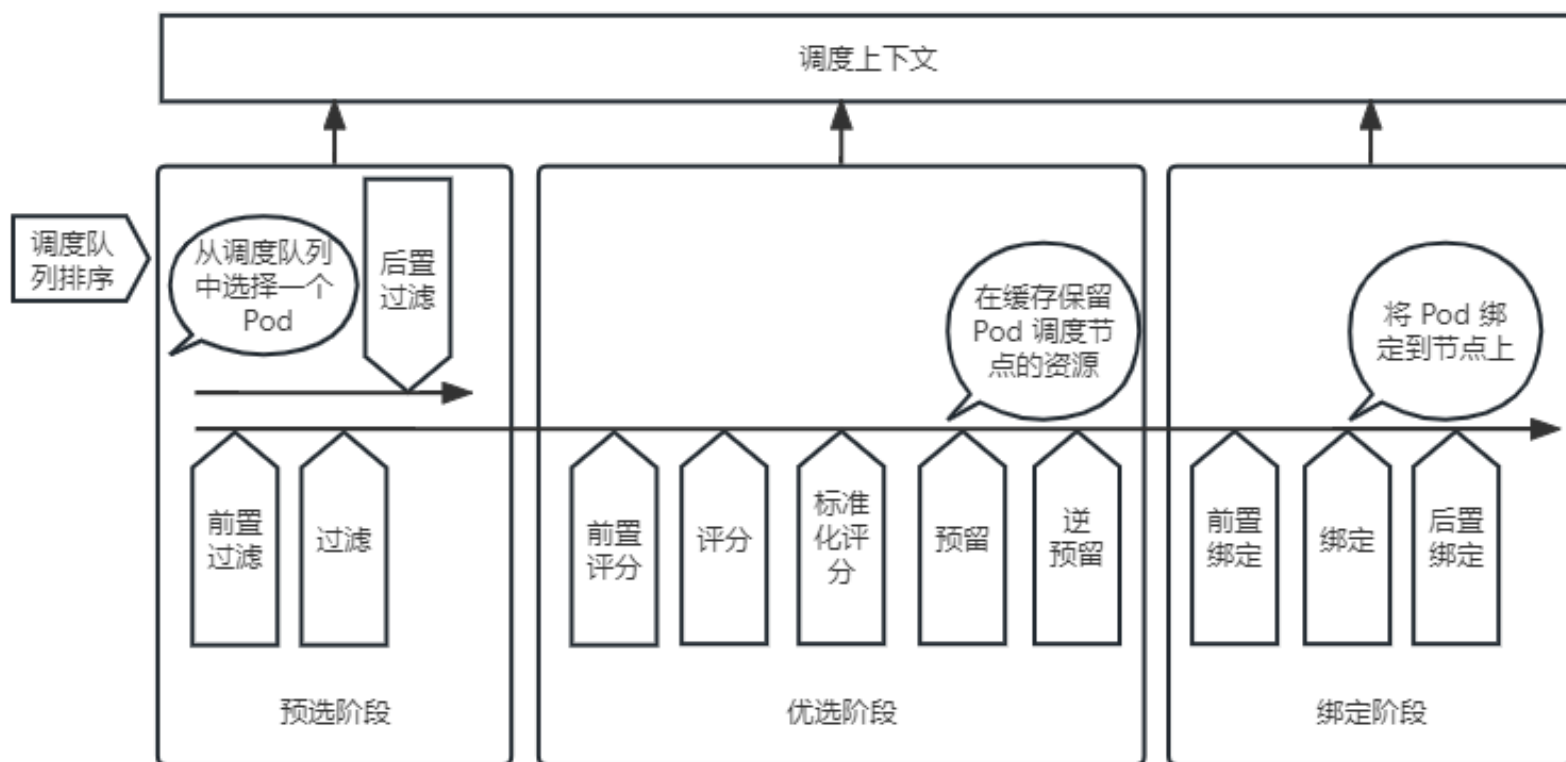
Kubernetes实现——调度器（1）

- 调度的最小单位是Pod； Pod 启动流程
 - 用户/工作负载创建 Pod
 - K8S的scheduler 选择最优节点，并将 Pod 与节点进行绑定
 - 节点的 Kubelet 拉取镜像、启动容器，负责Pod的管理
- 调度器负责监测集群中新创建并且没有被绑定节点的 Pod，然后根据 Pod 的配置以及内置的调度原则为其寻找一个最适合节点。
- 调度=调度决策+绑定（bind）
- 调度决策 = 预选（predicates） + 优选（priorities）



Kubernetes实现——调度器（2）

- 调度=预选（predicates）+ 优选（priorities）+ 绑定（bind）



Kubernetes实现——调度器（3）

•预选阶段

- 根据 Pod 的配置文件，调度器会过滤集群中每个节点，筛选出所有满足 Pod 需求的节点。
- 筛选出来的节点称为可调度节点，所有可调度节点放入一个可调度节点列表。
- 如果可调度节点列表为空，说明没有节点满足 Pod 的需求，Pod 无法被调度进入 Pending 状态。调度器会按一定的时间间隔重新调度 Pod，直到 Pod 被调度成功。

•优选阶段

- 根据设定的打分规则，给所有可调度节点打分
- 选出得分最高的节点

•绑定阶段

- 将 Pod 绑定到优选出来的节点；通知 api-server，绑定记录写入 etcd 数据库



Kubernetes实现——调度器（4）

- 调度器框架
 - 用于扩展调度器的核心功能。将调度器的功能分解为多个可插拔的模块，使得开发者可以根据自己的需求定制调度功能，并以插件的形式对调度器进行灵活的扩展和定制
- 每个可插拔的模块称为一个扩展点，一个调度器插件可以实现一个或多个扩展点，再通过配置集群中的调度器配置，选择启用插件的哪个扩展点
- Kubernetes 在调度 时在每个扩展点，按顺序调用实现并启用了该扩展点的插件



目录

- 容器基本概念
- Docker
 - 镜像
 - 容器和镜像的关系
- 容器编排
 - 动机、定义
 - 工作流程
 - 实现
 - 优点
- Kubernetes
 - 基础
 - 生态+实现
- 移动云容器服务案例



移动云容器服务详细介绍-容器服务 KCS

- 容器服务 KCS (Kubernetes Container Service)

提供了高性能高可靠的容器应用管理能力，支持容器化应用全生命周期管理，支持一键部署集群，并提供丰富的集群管理能力，简化云端容器化应用运行环境的搭建，为您提供完整的云端应用运行环境。

- 产品优势

- 简单易用：一键创建容器集群，一站式部署运维容器化应用，实现应用全生命周期管理。
- 安全可靠：集群控制面支持3 Master HA高可用，当其中某个Master节点故障时，集群依然可用，从而保障您的业务高可用。
- 开放兼容：Kubernetes服务提供商资质（KCSP），产品完全兼容Kubernetes/Docker社区原生版本。
- 集群规模：单个集群可管理节点数量达千级。



移动云容器服务详细介绍-容器服务 KCS

- 产品功能
 - 一站式部署和运维：为用户提供一键创建集群功能，简化 Kubernetes 集群部署和管理的工作。当前针对单用户单个资源池最多可订购5个集群，如需提升配额可通过提工单进行申请。容器服务目前支持多种集群类型，包括混合集群、GPU集群、边缘集群。
 - 丰富的应用调度策略：支持工作负载与节点以及负载间的多种亲和和反亲和调度策略，在高性能和高可靠中找到平衡点。



移动云容器服务详细介绍-容器服务 KCS

- 产品功能
 - 兼容原生Kubernetes和Docker：与社区最新版本保持紧密同步，完全兼容原生API和命令。
 - 高可用业务不中断：集群控制面HA，保障业务高可用。支持集群跨AZ部署，进一步提高了集群和业务的高可用。
 - 灵活的弹性伸缩策略：支持集群节点和工作负载2级的弹性伸缩，用户可自由组合策略以应对多变的突发浪涌。



原生 Kubernetes vs. KCS

	原生 Kubernetes	KCS (托管 Kubernetes)
集群部署	自己搭建	云平台自动化
运维管理	自己维护 Master 节点	云平台托管 Master, 用户只管 Node 和资源
升级维护	自己动手	一键升级
异构资源支持	需手动配置 GPU、NPU 等	原生支持 GPU、MIG、AI 加速调度
成本控制	靠经验优化	云平台有弹性伸缩、自动停机节省成本

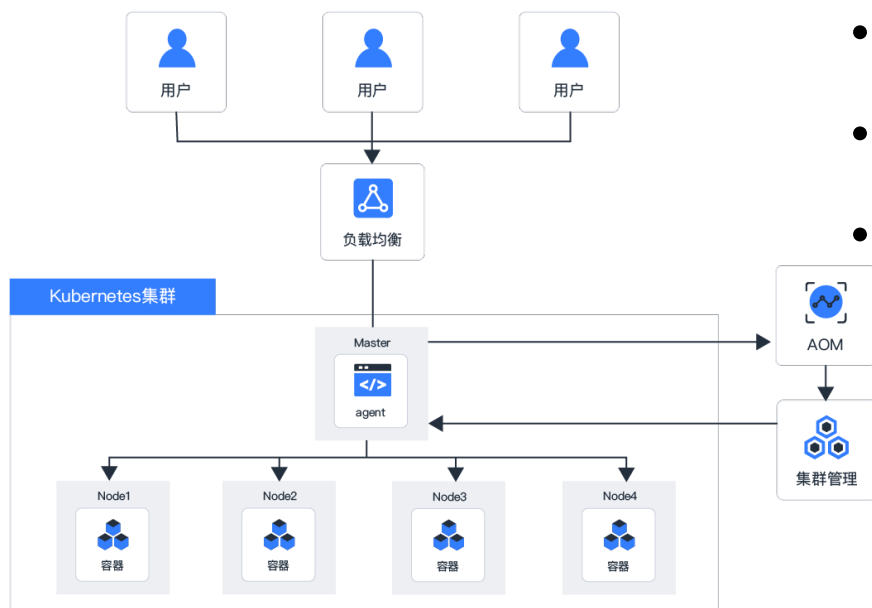


移动云容器服务KCS应用场景（2）

- 弹性伸缩

- 特点

- 用户自主预设弹性策略
- 计算资源随策略动态调整
- 资源利用率高，节省业务运行成本



电商客户遇到促销、限时秒杀等活动期间，访问量激增，需及时、自动扩展云计算资源。视频直播客户业务负载变化难以预测，需要根据CPU/内存使用率进行实时扩缩容。游戏客户每天中午12点及晚上18:00-23:00间需求增长，需要定时扩容。

移动云容器服务详细介绍-容器镜像服务 CIS

- 容器镜像服务 CIS (Container Image Service)
是一种简单易用、安全可靠的镜像管理服务，提供容器镜像的全生命周期管理，方便用户拉取、推送、删除镜像，并且提供大量移动云镜像、DockerHub镜像和用户公开镜像，方便用户使用。
- 产品优势
 - 操作便捷：提供命名空间管理、镜像管理等操作，均可在秒级范围内完成。
 - 海量镜像：提供移动云镜像、DockerHub镜像和用户公开镜像，方便用户使用。
 - 安全可靠：提供安全可靠的常用容器镜像，保障数据安全及访问合规。
 - 管理高效：可以通过控制台对容器镜像服务进行高效全生命周期的管理。



移动云容器服务详细介绍-容器镜像服务 CIS

- 产品功能
 - 镜像管理：提供镜像的全生命周期管理，支持推送、拉取、删除镜像，查看镜像层信息，自定义镜像描述和摘要等操作。
 - 空间管理：提供镜像的命名空间管理，方便用户对镜像进行逻辑上的隔离和管理，用户可选择性公开镜像。
 - 镜像收藏：用户可收藏镜像资源中的镜像，方便用户对其所关注的镜像进行管理和使用。
 - 镜像资源：为用户提供海量镜像资源，包括移动云镜像、DockerHub镜像和用户公开镜像，方便用户收藏和拉取。

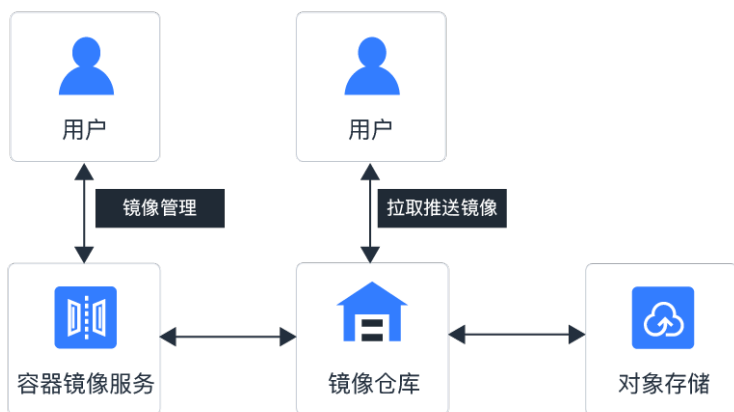


移动云容器镜像服务CIS应用场景（1）

- 镜像生命周期管理

- 特点

- 需自设密码登录镜像仓库进行推送和拉取等操作
- 提供操作指南，帮助用户使用镜像仓库存取镜像



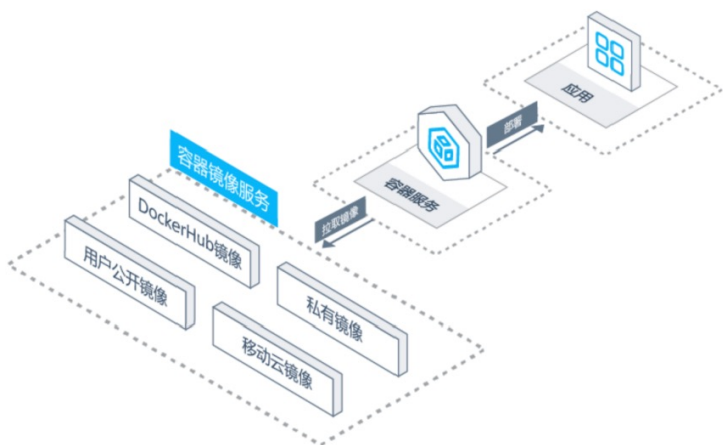
提供镜像推送、拉取、删除等全生命周期管理能力。容器镜像服务可为用户提供安全便捷的镜像管理功能，帮助用户更好的使用镜像。

移动云容器镜像服务CIS应用场景（2）

- 应用部署及更新

- 特点

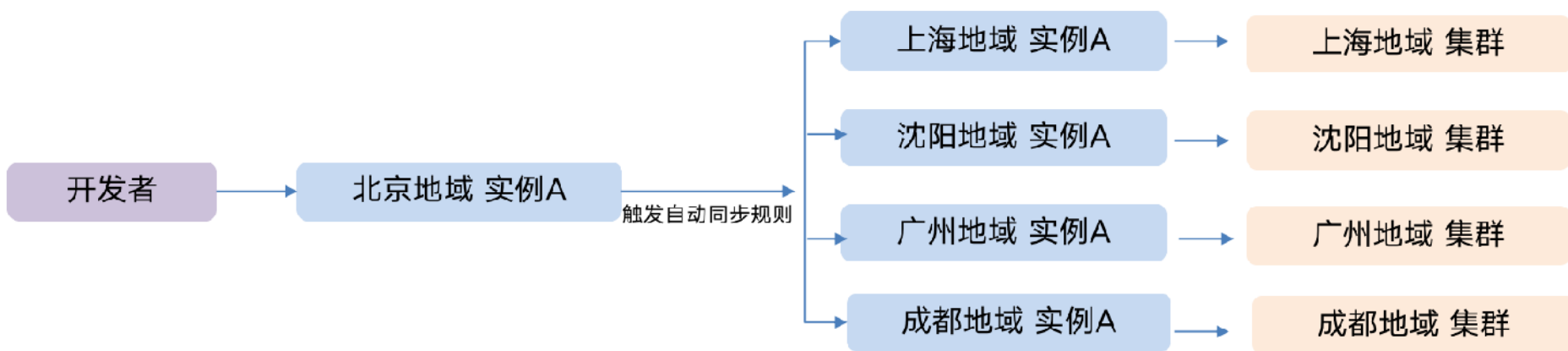
- 提供多种云原生资产来源供用户应用部署
- 可通过自定义触发器规则触发容器服务应用快速更新



配合容器服务，为用户提供多种云原生资产，手动或自定义触发器等
方式实现应用部署和更新。

移动云容器镜像服务CIS应用场景（3）

- 多区域镜像分发



容器镜像服务支持在多个区域部署多个实例，并可基于自定义规则实现自动数据同步，可实现全国容器业务就近拉取镜像，提升镜像拉取速度。

谢谢！



南京大學
NANJING UNIVERSITY